

# **ENPM665 Final Project Report**

## **Security Hardening for Cloud-Based E-Commerce Infrastructure**

### **A Multi-Layered Approach to Network Security and Data Protection**

**Author**

Ibaad Shaikh

**Course:** ENPM665 – Cloud Security

**Institution:** University of Maryland, College Park

## Contents

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>1 Executive Summary .....</b>                                       | <b>2</b>  |
| <b>2 Background: Connection to Midterm Assessment .....</b>            | <b>3</b>  |
| 2.1 Midterm Environment Overview .....                                 | 3         |
| 2.2 Relevant Midterm Findings for Network and Access Control.....      | 3         |
| 2.3 Relevant Midterm Findings for Data Protection and Encryption ..... | 4         |
| 2.4 Motivation for the Final Project Hardening Plan .....              | 4         |
| <b>3 Objectives and Key Improvements .....</b>                         | <b>5</b>  |
| 3.1 Focus Area 1: Network and Access Control Security .....            | 5         |
| 3.2 Focus Area 2: Data Protection and Encryption .....                 | 5         |
| <b>4 Risk Alignment with Midterm Findings .....</b>                    | <b>7</b>  |
| 4.1 Summary Mapping Table .....                                        | 7         |
| 4.2 Narrative Alignment per Improvement .....                          | 8         |
| <b>5 Implementation Plan .....</b>                                     | <b>10</b> |
| 5.1 Improvement 1: Security Group Hardening .....                      | 10        |
| 5.1.1 Initial Security Group State (Pre-Hardening) .....               | 10        |
| 5.1.2 Design Goals.....                                                | 10        |
| 5.1.3 Step-by-Step Configuration.....                                  | 10        |
| 5.1.4 Representative Security Group JSON .....                         | 12        |
| 5.1.5 Recommended SSH-Less Access .....                                | 13        |
| 5.2 Improvement 2: Network ACL (NACL) Restriction.....                 | 13        |
| 5.2.1 Initial NACL State.....                                          | 13        |
| 5.2.2 Design Goals.....                                                | 14        |
| 5.2.3 Step-by-Step Configuration.....                                  | 14        |
| 5.2.4 Application Entry via ALB .....                                  | 16        |
| 5.3 Improvement 3: S3 SSE-KMS Encryption and Enforcement Policy .....  | 17        |
| 5.3.1 Initial State .....                                              | 17        |
| 5.3.2 Design Goals.....                                                | 17        |
| 5.3.3 Step-by-Step Implementation .....                                | 18        |

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| 5.3.4 Verification of Encrypted Object Presence .....                           | 21        |
| 5.4 Improvement 4: IAM Least Privilege and Encryption Validation Workflow ..... | 21        |
| 5.4.1 Initial State .....                                                       | 21        |
| 5.4.2 Design Goals.....                                                         | 21        |
| 5.4.3 Step-by-Step Implementation .....                                         | 21        |
| <b>6 Validation and Testing .....</b>                                           | <b>24</b> |
| 6.1 Validation of Security Group Hardening.....                                 | 24        |
| 6.2 Validation of Network ACL Restrictions .....                                | 27        |
| 6.3 Validation of S3 SSE-KMS Encryption Enforcement .....                       | 29        |
| 6.4 Validation of IAM Least Privilege and Encryption Behavior .....             | 30        |
| <b>7 Impact Assessment .....</b>                                                | <b>32</b> |
| 7.1 Before-and-After Comparison .....                                           | 32        |
| 7.2 Qualitative Security Improvements .....                                     | 32        |
| 7.3 Trade-Offs and Considerations.....                                          | 33        |
| 7.4 Residual Risks .....                                                        | 34        |
| <b>8 Reflection &amp; Lessons Learned .....</b>                                 | <b>35</b> |
| 8.1 Challenges Faced .....                                                      | 35        |
| 8.2 Key Lessons Learned.....                                                    | 35        |
| 8.3 Future Improvements and Recommendations .....                               | 35        |

## 1. Executive Summary

In this project, we implement and validate four targeted security improvements to strengthen the confidentiality, integrity, and availability of a cloud-based e-commerce architecture.

This report presents a security hardening plan for a cloud-based e-commerce environment built on AWS. Building on the vulnerabilities and risks identified in our midterm assessment, we selected two primary focus areas: **Network and Access Control Security** and **Data Protection and Encryption**.

For **Network and Access Control Security**, we observed that the midterm deployment exposed an EC2-hosted OWASP Juice Shop instance directly to the internet with overly permissive security groups, unrestricted SSH access, and default-allow network ACLs. In response, we implemented two

key improvements: *Security Group Hardening* and *Network ACL Restriction*. Together, these changes reduce the external attack surface, enforce least privilege for both inbound and outbound traffic, and introduce a subnet-level defense-in-depth layer.

For **Data Protection and Encryption**, the midterm showed a lack of enforced encryption at rest and overly permissive access to data. To address this, we implemented *S3 server-side encryption with AWS KMS (SSE-KMS) enforced by bucket policy* and an *IAM least privilege validation workflow* that uses a restricted IAM user to validate encryption and access controls in practice.

Across both focus areas, we implemented four improvements, each directly tied to specific midterm findings. We provide detailed implementation steps, AWS CLI commands, security policies, simulated evidence (with embedded screenshots), validation procedures, and a before-and-after impact assessment. The end result is a more defensible, monitored, and resilient architecture that aligns with cloud security best practices while remaining realistic for an academic lab environment.

## 2. Background: Connection to Midterm Assessment

### 2.1 Midterm Environment Overview

In the midterm project, our team assessed the security posture of an AWS-hosted OWASP Juice Shop deployment. The environment used:

- A single EC2 instance in a public subnet.
- Security groups with open inbound rules for SSH (22) and application ports.
- Default network ACLs allowing all inbound and outbound traffic.
- Local storage and data handling without explicit enforcement of encryption.

The goal of the midterm was to identify vulnerabilities across compute, network, data, and disaster recovery domains. The final deliverable consisted of multiple assessment sections, and the findings from that work directly guided the hardening measures in this final project.

### 2.2 Relevant Midterm Findings for Network and Access Control

The midterm network and host assessment highlighted several critical issues that directly motivated our Network and Access Control improvements:

- **Midterm Finding #1 (Application Exposure):** The OWASP Juice Shop application was reachable over plain HTTP on port 3000 from any public IP. This meant that a deliberately insecure application was fully exposed to the internet.

- **Midterm Finding #2 (SSH Exposure):** The security group associated with the EC2 instance allowed SSH (port 22) from 0.0.0.0/0. This created a high risk of brute-force or credential-stuffing attacks against administrative access.
- **Midterm Finding #3 (Flat Network Architecture):** The EC2 instance was placed in a public subnet with a route to the internet gateway and default-allow network ACLs, resulting in a flat network with no meaningful segmentation or subnet-level controls.
- **Midterm Finding #4 (Unrestricted Outbound Traffic):** Default security group and NACL configurations allowed all outbound traffic, making it trivial for a compromised instance to exfiltrate data or communicate with external command-and-control servers.

These findings clearly indicated that network boundaries and access controls were too permissive and that multiple layers of protection were missing.

### 2.3 Relevant Midterm Findings for Data Protection and Encryption

The midterm data security assessment focused on how sensitive data was stored and protected. Key findings included:

- **Midterm Finding #5 (Lack of Enforced Encryption):** The environment did not enforce encryption for data-at-rest in storage services. There was no bucket policy or configuration requiring server-side encryption for S3 objects.
- **Midterm Finding #6 (Potential for Plaintext Data Exposure):** Because encryption was not enforced, sensitive data could be uploaded in plaintext, and there was no mechanism guaranteeing that all objects would be protected by strong cryptographic controls.
- **Midterm Finding #7 (Overly Permissive Data Access):** IAM permissions for data access were not clearly scoped, leaving the potential for broader-than-necessary read or write access to critical data.

Collectively, these issues posed a high risk to the confidentiality and integrity of stored data, particularly if an attacker gained access to storage or credentials.

### 2.4 Motivation for the Final Project Hardening Plan

This final project moves beyond identifying weaknesses and focuses on implementing targeted hardening controls. The four improvements we selected are intentionally designed to:

- Directly remediate specific midterm findings in a measurable way.
- Leverage AWS-native services such as security groups, NACLs, S3, KMS, and IAM.
- Provide clear validation steps that demonstrate the impact of the changes.

- Serve as building blocks for a more robust, production-like security posture.

The rest of this report documents the objectives, implementation details, validation, and impact of these changes.

### 3. Objectives and Key Improvements

#### 3.1 Focus Area 1: Network and Access Control Security

##### *Objective*

The objective for the Network and Access Control Security focus area is to **reduce the external attack surface and enforce least privilege for network flows at both the instance and subnet levels**. This includes restricting administrative access, controlling application exposure, and limiting outbound traffic to only what is necessary.

##### *Selected Improvements*

We implemented the following two key improvements under this focus area:

**1.1 Security Group Hardening What will be done:** Remove public SSH access, restrict inbound application traffic to only required ports and paths, and limit outbound traffic to a minimal set of protocols (DNS and HTTPS). **AWS services used:** Amazon EC2, Security Groups, AWS Systems Manager (recommended for future SSH-less access). **Midterm risks addressed:** Directly mitigates Midterm Finding #1 (application exposure on port 3000), Midterm Finding #2 (SSH open to 0.0.0.0/0), and Midterm Finding #4 (unrestricted outbound traffic).

**1.2 Network ACL (NACL) Restriction What will be done:** Replace the default-allow NACL with a restrictive NACL that enforces allowed inbound and outbound flows at the subnet level. **AWS services used:** Amazon VPC, Network ACLs. **Midterm risks addressed:** Directly mitigates Midterm Finding #3 (flat network architecture with no subnet-level enforcement) and further reinforces the control introduced for Midterm Finding #4 (unrestricted outbound traffic).

#### 3.2 Focus Area 2: Data Protection and Encryption

##### *Objective*

The objective for the Data Protection and Encryption focus area is to **ensure that sensitive data stored in AWS is always encrypted at rest using strong, centrally managed keys, while enforcing least privilege for data access and validating that these protections work in practice**.

##### *Selected Improvements*

We implemented the following two key improvements under this focus area:

##### **2.1 S3 Server-Side Encryption with AWS KMS (SSE-KMS) and Enforcement Policy**

**What will be done:** Create a customer managed KMS key, configure an S3 bucket to use SSE-KMS by default, and attach a bucket policy that denies any non-encrypted uploads. **AWS services used:** Amazon S3, AWS Key Management Service (KMS). **Midterm risks addressed:** Directly mitigates Midterm Finding #5 (lack of enforced encryption) and Midterm Finding #6 (potential for plaintext data exposure).

**2.2 IAM Least Privilege and Encryption Validation Workflow** **What will be done:** Create a restricted IAM user for testing, validate both positive and negative test cases for encrypted access, and verify that unauthorized operations are blocked. **AWS services used:** AWS Identity and Access Management (IAM), Amazon S3, AWS KMS. **Midterm risks addressed:** Directly mitigates Midterm Finding #7 (overly permissive data access) and confirms that the encryption and access controls are effective.

#### 4. Risk Alignment with Midterm Findings

This section explicitly maps each of the four security improvements in this project back to the midterm findings they remediate. The goal is to show clear traceability from identified risks to implemented controls.

##### 4.1 Summary Mapping Table

| Improvement               | Relevant Midterm Finding(s)                                                                                                                              | How the Improvement Remediates the Risk                                                                                                                                                                                                                                                                    |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Security Hardening        | Group Finding #1: Application exposed on port 3000 to internet.<br>Finding #2: SSH (22) open to 0.0.0.0/0.<br>Finding #4: Unrestricted outbound traffic. | Inbound SSH from the internet is removed entirely, eliminating a major brute-force and credential-stuffing vector. Application ports are restricted to only necessary traffic paths, and outbound rules are reduced to DNS and HTTPS, minimizing opportunities for data exfiltration and C2 communication. |
| Network ACL Restriction   | Finding #3: Flat network architecture with default-allow NACL.<br>Finding #4: Unrestricted outbound traffic.                                             | The default NACL is replaced with a restrictive ACL that explicitly allows only required inbound and outbound ports. This means that even if a security group becomes misconfigured, subnet-level controls prevent unauthorized traffic from entering or leaving the subnet.                               |
| S3 Encryption Enforcement | Finding #5: Lack of enforced encryption at rest.<br>Finding #6: Potential for plaintext data exposure.                                                   | The S3 bucket is configured with default SSE-KMS and a bucket policy that denies any PutObject request that does not specify encryption. This guarantees that all objects written to the bucket are encrypted with a managed KMS key, closing the gap on plaintext storage.                                |
| Improvement               | Relevant Midterm Finding(s)                                                                                                                              | How the Improvement Remediates the Risk                                                                                                                                                                                                                                                                    |

---

|                                   |           |                                                                                                                     |                                                                                                                                                                                                                                                                                              |
|-----------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IAM Privilege Validation Workflow | Least and | Finding #7: Overly permissive data access.<br>Indirectly supports Finding #5 and #6 by validating encryption usage. | A restricted IAM user is created with minimal permissions. Validation tests confirm that this user can read, but not write or administer sensitive resources. AccessDenied responses for unauthorized operations demonstrate that data is only accessible to properly authorized principals. |
|-----------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

#### 4.2 Narrative Alignment per Improvement

##### *Security Group Hardening → Findings #1, #2, #4*

In the midterm, the EC2 instance listened on both SSH (22) and application port 3000, with inbound rules permitting 0.0.0.0/0. This configuration made the system vulnerable to direct exploitation of the application and brute-force attacks against SSH.

The Security Group Hardening improvement removes external SSH access, leaving the instance accessible only through safer mechanisms (such as Session Manager) and restricts inbound rules to necessary application flows. Outbound rules are reduced to DNS and HTTPS, closing the gap of unrestricted data exfiltration.

##### *Network ACL Restriction → Findings #3, #4*

The midterm documented a flat network architecture where the subnet used the default NACL that allowed any inbound or outbound traffic. This meant that even if security groups were tightened, there was no safety net at the subnet boundary.

By introducing a custom NACL that only allows specific inbound and outbound traffic, we implement a second layer of defense. If a security group change accidentally opens a port, the NACL still restricts unauthorized traffic, directly addressing the lack of subnet-level enforcement.

##### *S3 SSE-KMS Enforcement → Findings #5, #6*

The midterm identified that encryption was not enforced for data stored in S3. This created a risk that sensitive information could be stored unencrypted, whether by mistake or through malicious uploads.

With default SSE-KMS enabled and a bucket policy that denies any non-encrypted PutObject, encryption becomes mandatory. Even a misconfigured or malicious client cannot store unencrypted objects in the bucket, directly remediating the identified risk.

##### *IAM Least Privilege and Validation → Finding #7*

Overly permissive access to data was another midterm concern. Without clear least-privilege controls, a compromise of any credential could yield broad access to stored data.

By creating a dedicated, restricted IAM user and validating both positive (allowed) and negative (denied) actions, we confirm that access has been narrowed to only what is necessary. This reduces the blast radius of any credential compromise and enforces concrete boundaries for data access.

## 5. Implementation Plan

This section provides step-by-step implementation details for each of the four selected improvements. For the Network and Access Control focus area, we embed the actual AWS console screenshots captured during implementation. For Data Protection and Encryption, we embed the screenshots extracted from the S3 encryption report and validation & testing document.

### 5.1 Improvement 1: Security Group Hardening

#### 5.1.1 Initial Security Group State (Pre-Hardening)

At the start of this improvement, the EC2 instance's security group was configured as follows:

- Inbound:
  - TCP 22 (SSH) from 0.0.0.0/0 – TCP 3000 (application) from 0.0.0.0/0
- Outbound:
  - All traffic allowed to 0.0.0.0/0

These settings reflected the midterm findings and were intentionally permissive to demonstrate risk.

#### 5.1.2 Design Goals

The Security Group Hardening implementation had the following goals:

- Remove direct SSH access from the internet.
- Retain application reachability via the intended public entry point (e.g., ALB or carefully controlled IP ranges).
- Reduce outbound rules so that only DNS queries and HTTPS traffic are allowed.

#### 5.1.3 Step-by-Step Configuration

##### Step 1: Remove SSH (22) from Inbound Rules

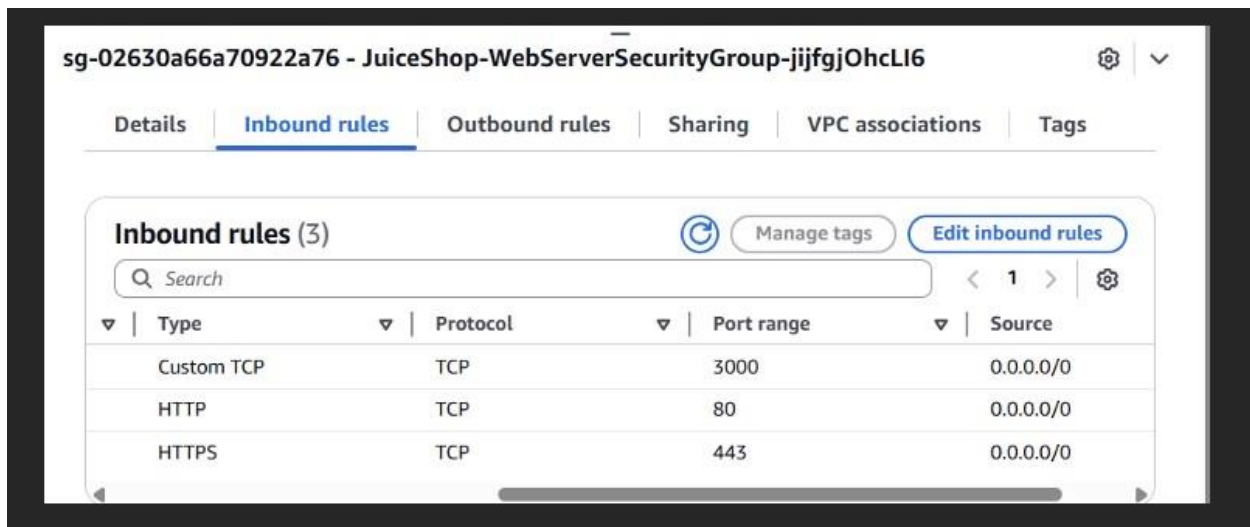
From the AWS Management Console:

1. Navigate to *EC2* → *Security Groups*.
2. Select the security group associated with the Juice Shop EC2 instance.
3. Open the *Inbound rules* tab.

4. Locate the rule: Type: SSH, Port: 22, Source: 0.0.0.0/0.
5. Delete this rule and save changes.

**Step 2: Ensure Only Required Application Ports Are Open** In the same inbound rules view:

- Retain or add rules for:
  - HTTP (80) from 0.0.0.0/0 (if using HTTP front-end).
  - HTTPS (443) from 0.0.0.0/0 (preferred).
  - Application port (3000) only if absolutely necessary, ideally restricted to the ALB or specific IP ranges.



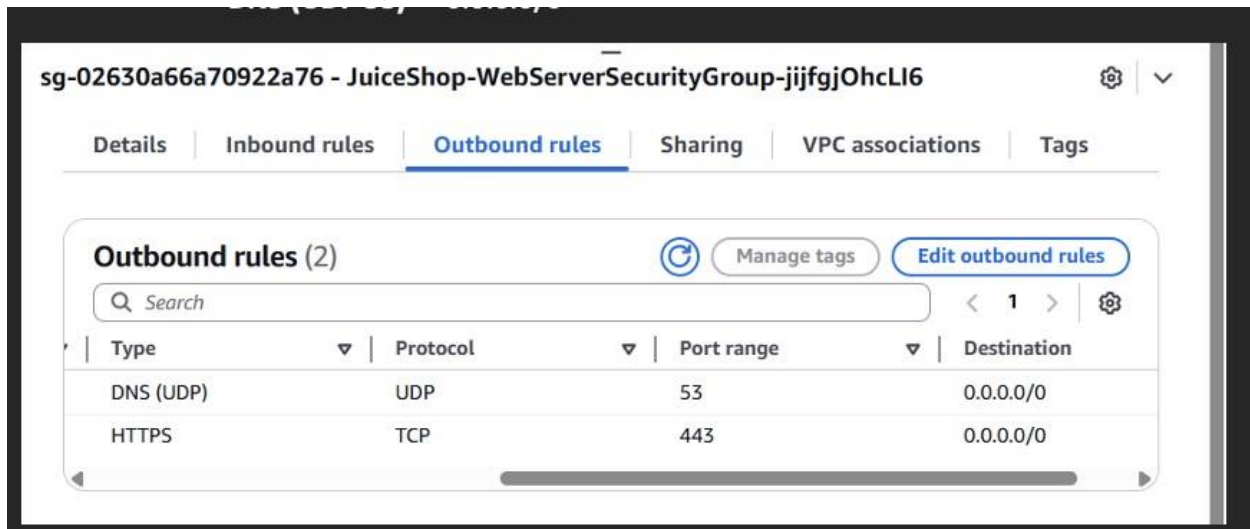
**Figure 1:** Security Group inbound rules after hardening (HTTP/HTTPS and application port only).

### Step 3: Restrict Outbound Rules

From the *Outbound rules* tab:

1. Remove the default *All traffic* rule.
2. Add a rule for DNS:
  - Type: Custom UDP
  - Port range: 53 • Destination: 0.0.0.0/0
3. Add a rule for HTTPS:
  - Type: HTTPS

- Port: 443
- Destination: 0.0.0.0/0



**Figure 2:** Security Group outbound rules restricted to DNS and HTTPS.

#### 5.1.4 Representative Security Group JSON

For reproducibility, the final security group configuration can be represented as:

```
{
  "GroupId": "sg-xxxxxxxxxxxxxx",
  "IpPermissions": [
    {
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
    },
    {
      "IpProtocol": "tcp",
      "FromPort": 443,
      "ToPort": 443,
      "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
    },
    {
      "IpProtocol": "tcp",
      "FromPort": 3000,
      "ToPort": 3000,
      "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
    }
  ]
}
```

```
],  
  "IpPermissionsEgress": [  
    {  
      "IpProtocol": "udp",  
      "FromPort": 53,  
      "ToPort": 53,  
      "IpRanges": [{"CidrIp": "0.0.0.0/0"}]  
    },  
    {  
      "IpProtocol": "tcp",  
      "FromPort": 443,  
      "ToPort": 443,  
      "IpRanges": [{"CidrIp": "0.0.0.0/0"}]  
    }  
  ]  
}
```

### 5.1.5 Recommended SSH-Less Access

Although SSH has been removed from the security group, administrators still need a secure way to access the instance. We configured *AWS Systems Manager Session Manager* for shell access without exposing SSH to the public internet.



**Figure 3:** Session Manager access to the EC2 instance as ssm-user.

## 5.2 Improvement 2: Network ACL (NACL) Restriction

### 5.2.1 Initial NACL State

Initially, the subnet used the default NACL, which included:

- Inbound: Allow all protocols from 0.0.0.0/0.
- Outbound: Allow all protocols to 0.0.0.0/0.

This configuration offered no subnet-level enforcement and was fully permissive.

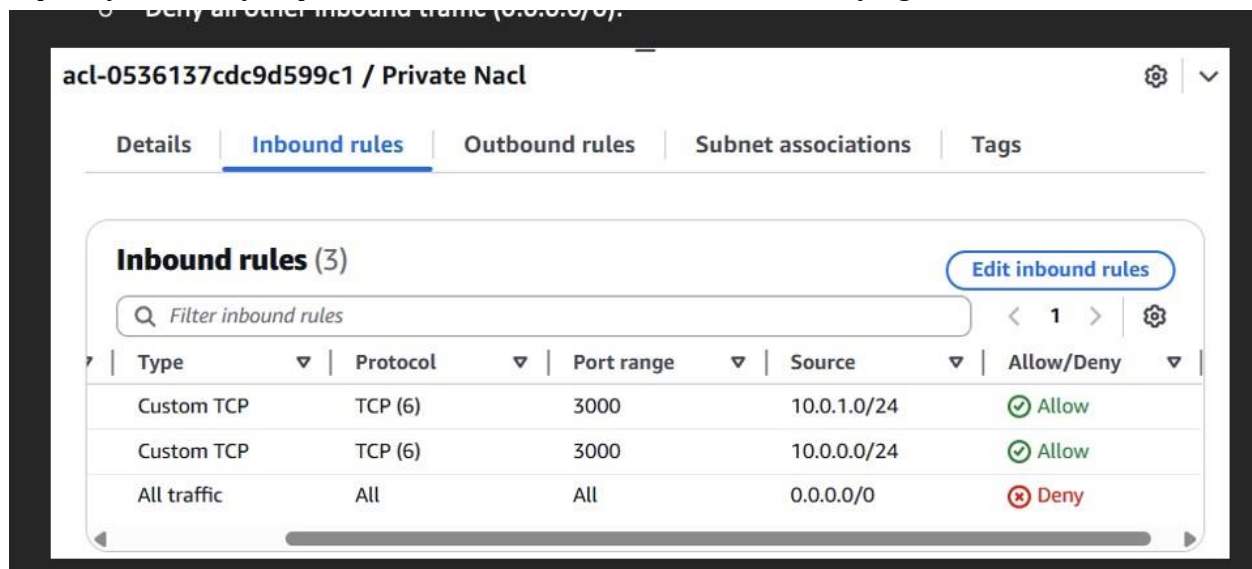
### 5.2.2 Design Goals

The NACL Restriction improvement aimed to:

- Permit inbound HTTP/HTTPS (or ALB traffic) to the public subnet.
- Permit only necessary inbound traffic from the ALB to the private subnet.
- Permit outbound DNS and HTTPS traffic from the private subnet.
- Deny all other inbound and outbound traffic.

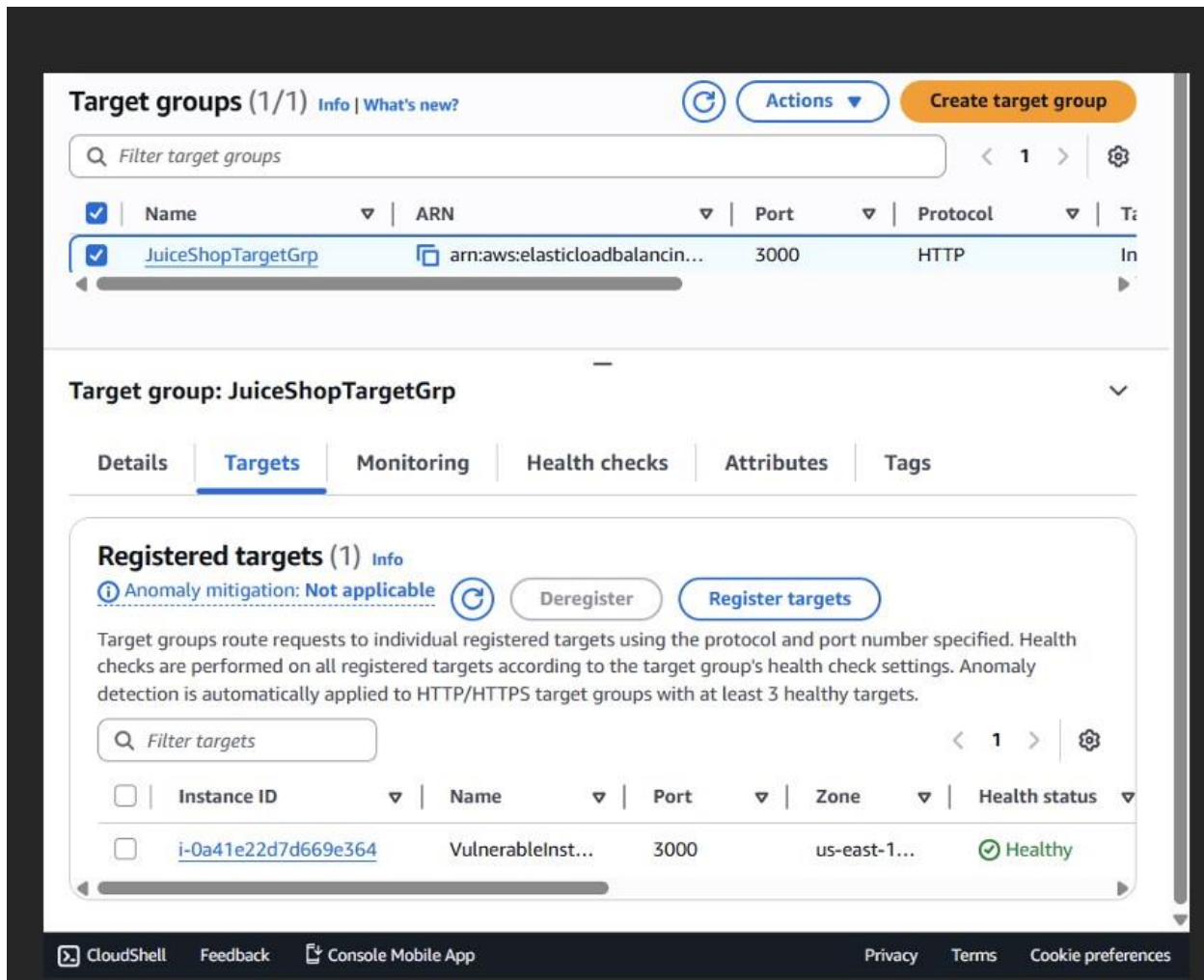
### 5.2.3 Step-by-Step Configuration

**Step 1: Configure Private Subnet NACL Inbound Rules** The private subnet NACL was updated to explicitly allow only required traffic from the ALB subnets while denying all other inbound traffic.



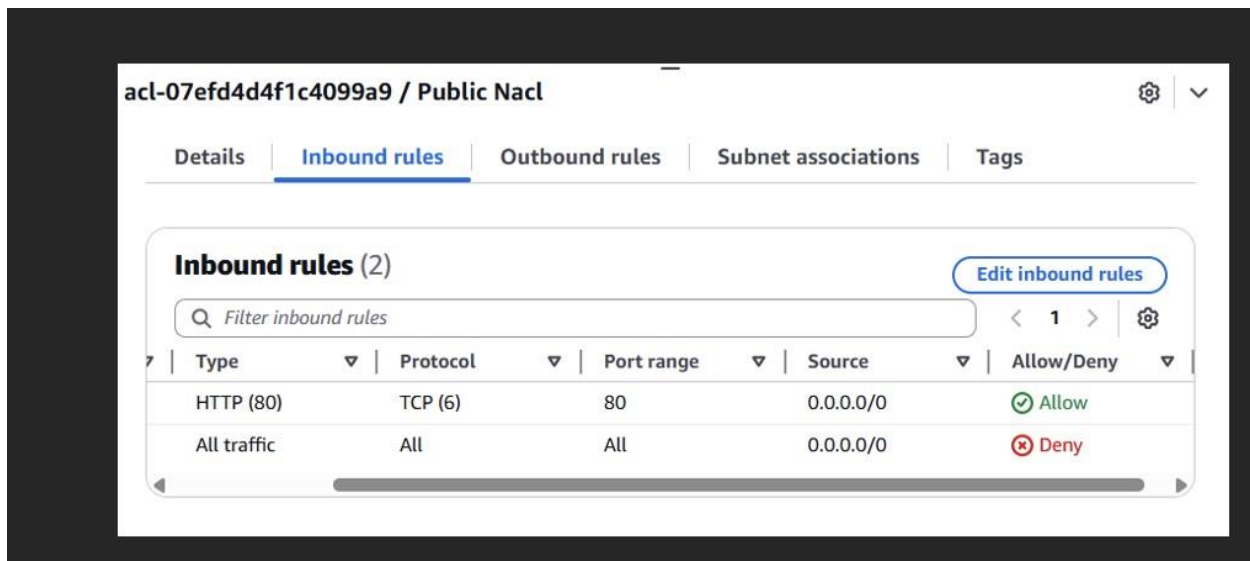
**Figure 4:** Private subnet NACL inbound rules allowing only application traffic from ALB subnets.

**Step 2: Configure Public Subnet NACL Inbound Rules** The public subnet NACL was configured to allow inbound HTTP from the internet while denying all other inbound traffic, ensuring that only port 80 is reachable externally.



**Figure 5:** Public subnet NACL inbound rules allowing only HTTP and denying all other traffic.

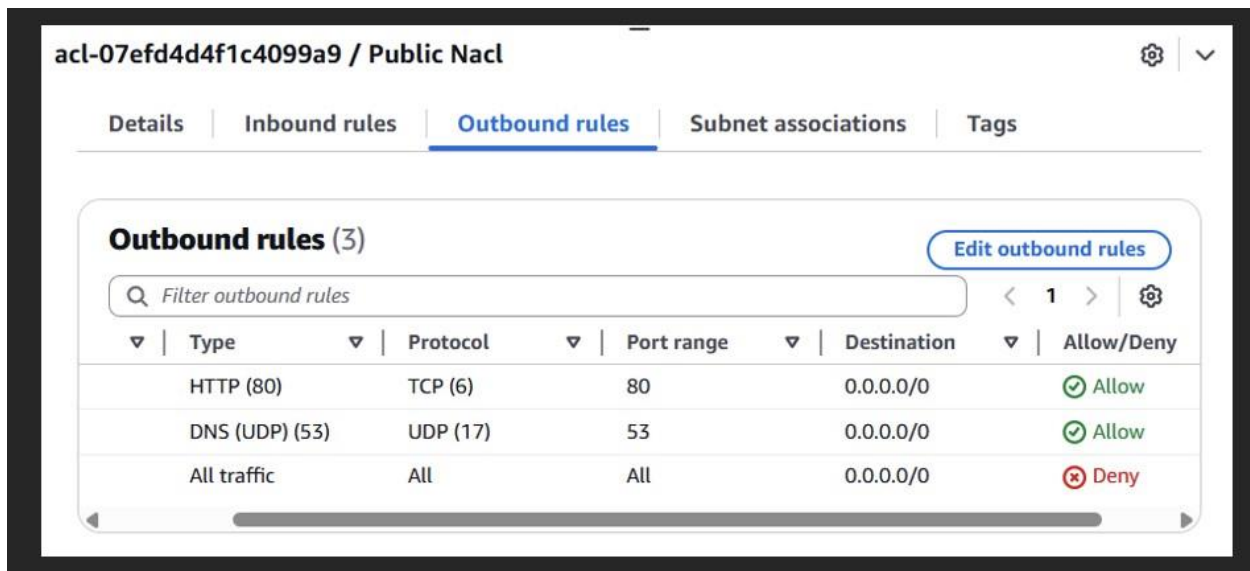
**Step 3: Configure Public Subnet NACL Outbound Rules** Outbound rules were tightened so that only HTTP and DNS traffic are allowed from the public subnet to the internet, with a deny-all rule as a safety net.



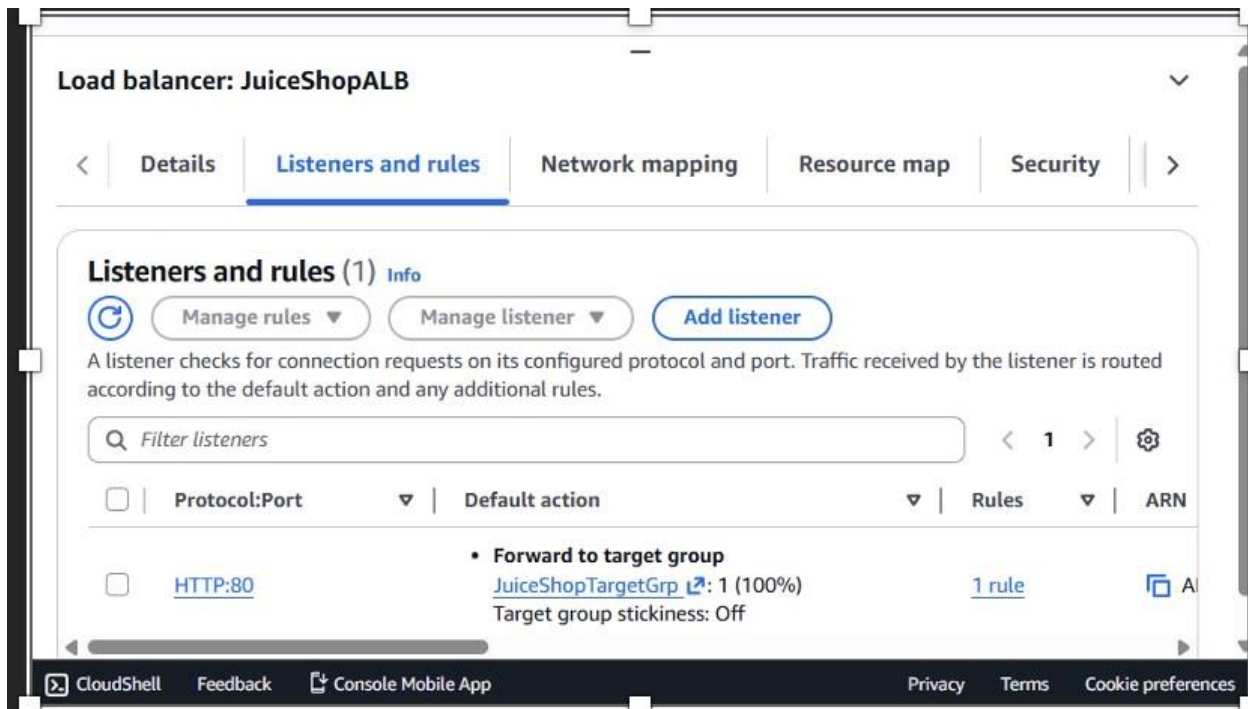
**Figure 6:** Public subnet NACL outbound rules restricted to HTTP and DNS.

#### 5.2.4 Application Entry via ALB

To ensure that users still reach the Juice Shop application through a controlled entry point, we configured an Application Load Balancer (ALB) that forwards HTTP requests to the target group containing the hardened EC2 instance.



**Figure 7:** ALB listener routing HTTP traffic to the Juice Shop target group.



**Figure 8:** Juice Shop target group with the registered EC2 instance on port 3000.

Together, these NACL changes and ALB configuration enforce a clear, auditable network path: internet clients reach the ALB, which forwards traffic to the instance in a protected subnet that is governed by restrictive NACLs and hardened security groups.

### 5.3 Improvement 3: S3 SSE–KMS Encryption and Enforcement Policy

#### 5.3.1 Initial State

In the midterm assessment, there was no guarantee that objects stored in S3 would be encrypted. There was no default encryption configured and no bucket policy designed to deny unencrypted uploads.

#### 5.3.2 Design Goals

- Ensure all new objects stored in the target bucket are encrypted using SSE–KMS.
- Prevent any non-encrypted uploads from being accepted.
- Use a customer managed KMS key for fine-grained control and auditing.

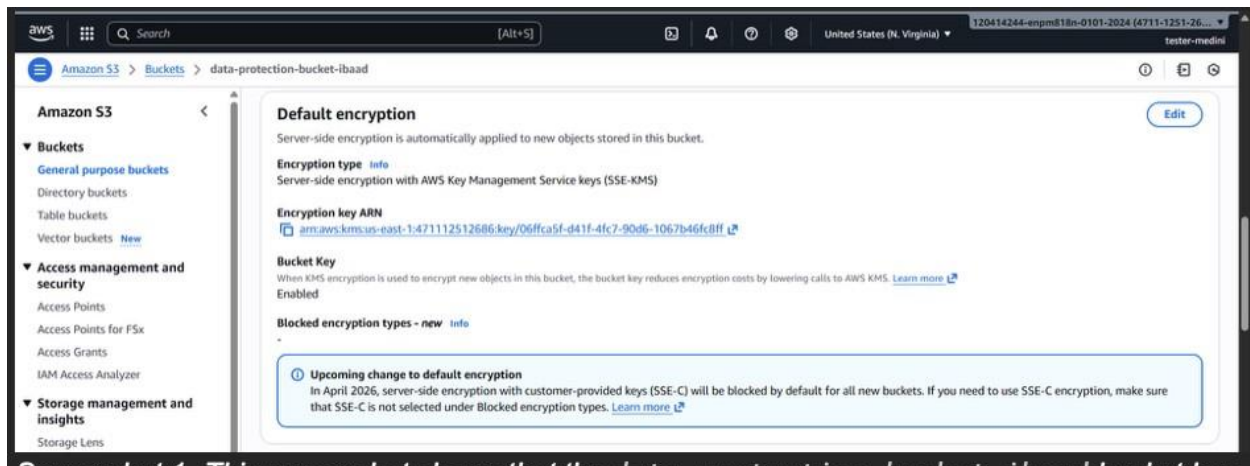
### 5.3.3 Step-by-Step Implementation

#### Step 1: Create a Customer-Managed KMS Key

1. Navigate to *KMS* → *Customer managed keys*.
2. Create a new symmetric key.
3. Set an alias such as *project-s3-kms-key*.
4. Enable automatic key rotation.
5. Assign key administrators and users appropriately.

#### Step 2: Create or Configure the S3 Bucket with Default SSE–KMS

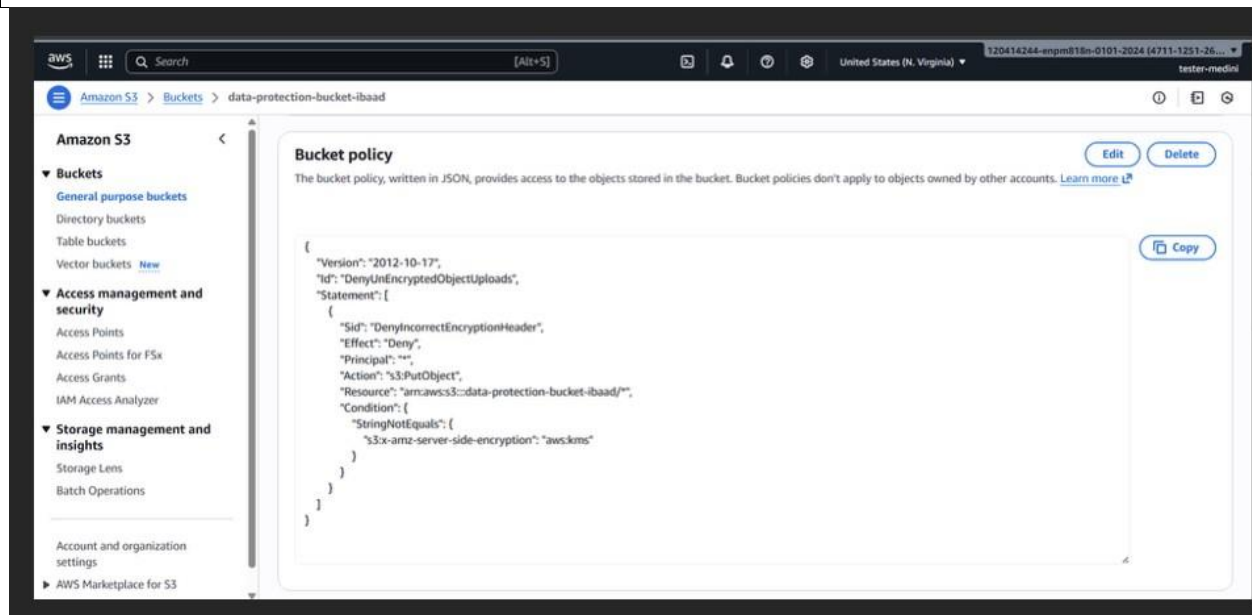
1. Navigate to *S3* → *Buckets*.
2. Create a new bucket (e.g., *data-protection-bucket-<team>*).
3. Enable “Block all public access”.
4. Enable versioning to allow rollback of accidental changes.
5. Under *Default encryption*, choose AWS KMS and select the *project-s3-kms-key*.



**Figure 9:** S3 bucket properties showing default server-side encryption configured with AWS KMS.

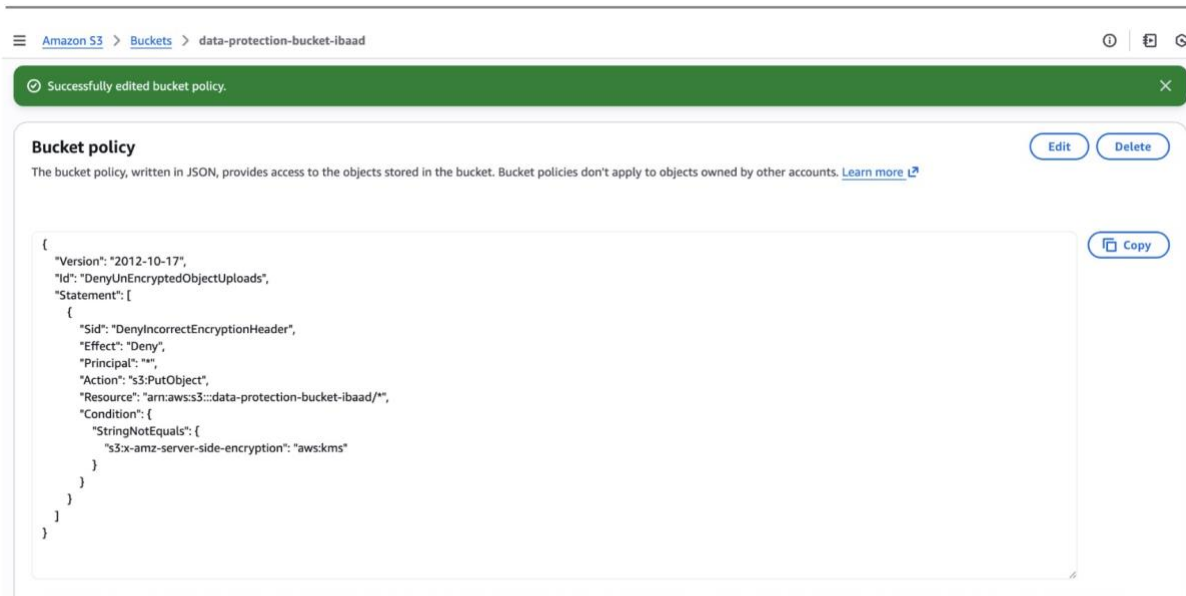
**Step 3: Attach a Bucket Policy Enforcing Encryption** Attach a policy that denies any PutObject request that does not include SSE–KMS:

```
{
  "Version": "2012-10-17",
  "Id": "DenyUnEncryptedObjectUploads",
  "Statement": [
    {
      "Sid": "DenyIncorrectEncryptionHeader",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::data-protection-bucket-team/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "aws:kms"
        }
      }
    }
  ]
}
```



**Figure 10:** Bucket policy JSON enforcing SSE–KMS for all PutObject operations.

After saving, the console confirms that the policy has been successfully edited.



**Figure 11:** Confirmation banner in the S3 console indicating the bucket policy was successfully applied.

**Step 4: Test Encrypted Upload via AWS CLI Using an IAM entity with appropriate permissions:**

```
aws s3 cp testfile.txt s3://data-protection-bucket-team/ \
--sse aws:kms \
--sse-kms-key-id <KMS_KEY_ID>
```

We also compared this to an upload that omitted the encryption flags to validate the effect of the policy.

```
baadshaikh@Mac ~ % aws s3 cp testfile.txt s3://data-protection-bucket-ibaad/ \
--sse aws:kms \
--sse-kms-key-id arn:aws:kms:us-east-1:471112512686:key/06ffca5f-d41f-4fc7-90d
-1067b46fc8ff

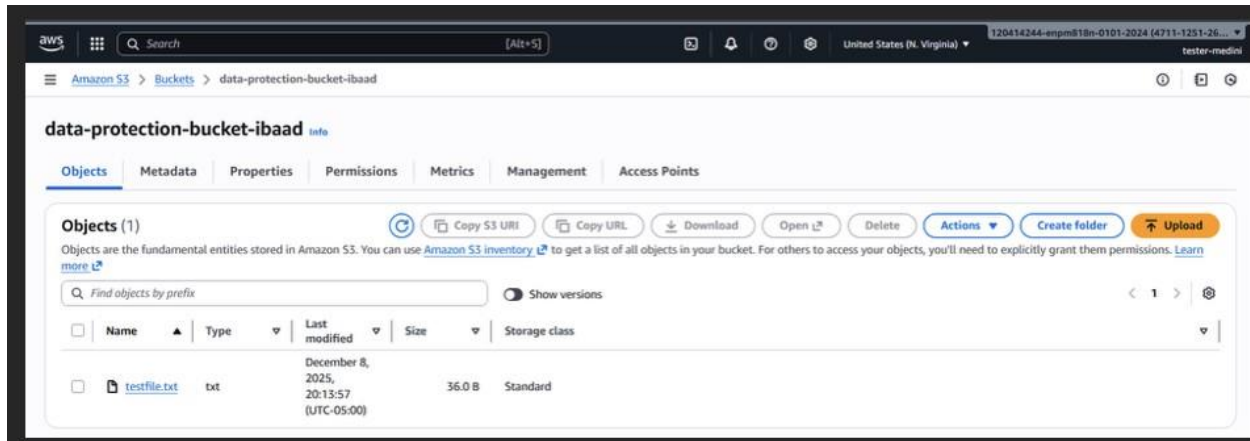
upload: ./testfile.txt to s3://data-protection-bucket-ibaad/testfile.txt
baadshaikh@Mac ~ % aws s3 cp testfile.txt s3://data-protection-bucket-ibaad/

upload failed: ./testfile.txt to s3://data-protection-bucket-ibaad/testfile.txt
an error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:iam::471112512686:root is not authorized to perform: s3:PutObject on resource: "arn:aws:s3:::data-protection-bucket-ibaad/testfile.txt" with an explicit deny in a resource-based policy
baadshaikh@Mac ~ %
```

**Figure 12:** CLI test: encrypted upload succeeds, while a follow-up unencrypted upload is denied with AccessDenied.

### 5.3.4 Verification of Encrypted Object Presence

After uploading, we confirmed that the object exists in the bucket and is stored under the protected path.



**Figure 13:** S3 bucket objects listing showing testfile.txt successfully stored in the encrypted bucket.

## 5.4 Improvement 4: IAM Least Privilege and Encryption Validation Workflow

### 5.4.1 Initial State

Before this improvement, encryption and bucket policies could be configured, but there was no systematic way to validate that these controls behaved as intended from the perspective of a non-privileged user.

### 5.4.2 Design Goals

- Validate that unencrypted uploads are denied.
- Validate that a restricted IAM user cannot perform administrative actions.
- Validate that an authorized user can read encrypted objects successfully.

### 5.4.3 Step-by-Step Implementation

#### Step 1: Create a Restricted IAM User

1. Navigate to *IAM* → *Users*.

2. Create a user dedicated to testing encryption and access control (for example, tester-encryption).
3. Attach only the minimum policies required (read access to the S3 bucket and decrypt permission on the specific KMS key, if needed).

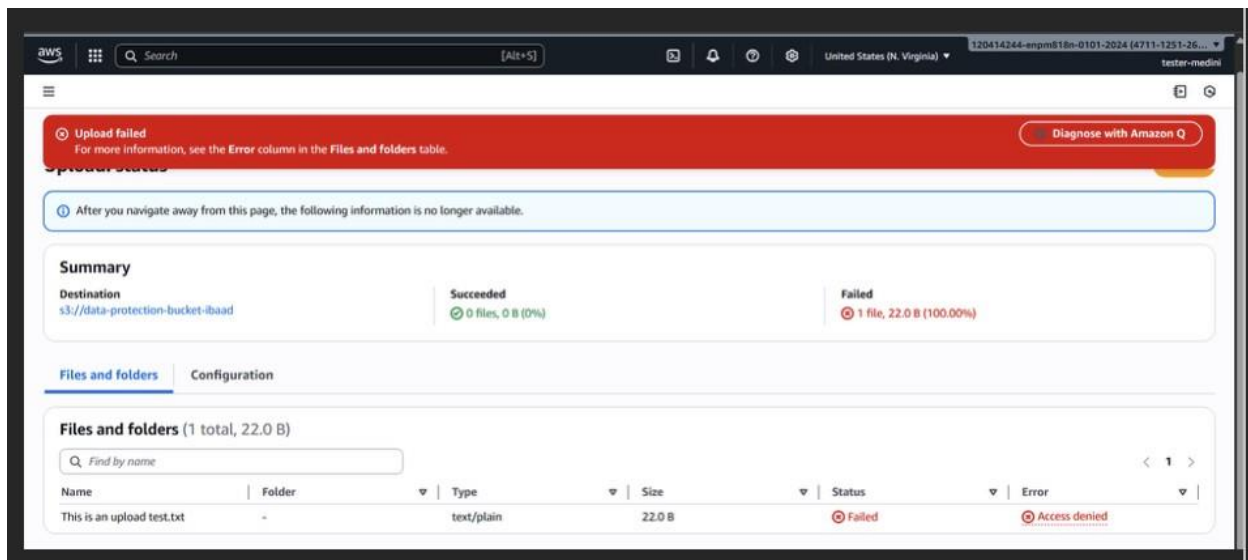
### Step 2: Login as the Restricted User

- Use the provided AWS sign-in URL for the account.
- Log in with the restricted tester credentials.

### Step 3: Attempt Unencrypted or Unauthorized Upload (Negative Test)

- In the S3 console, navigate to the encrypted bucket.
- Attempt to upload a file via the console without specifying encryption (or with default settings).

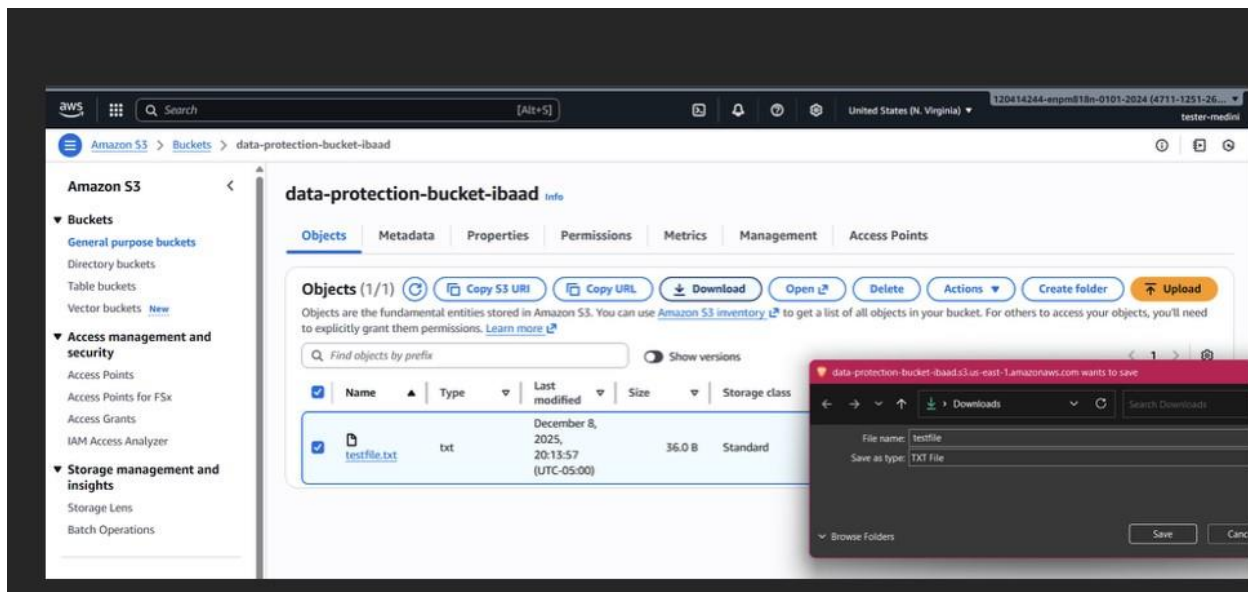
The expected result is that the upload fails due to the bucket policy or IAM permissions.



**Figure 14:** Upload attempt to the protected bucket failing with Access denied, confirming that unencrypted or unauthorized writes are blocked.

### Step 4: View and Download Encrypted Object (Positive Validation)

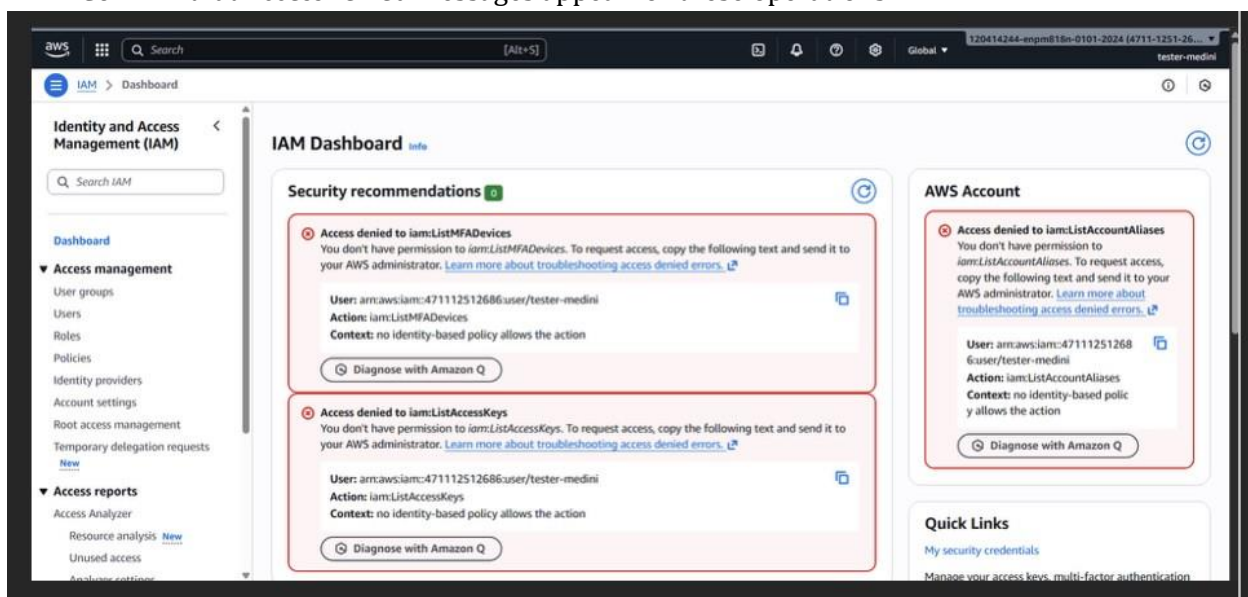
- Using the restricted account, view the existing encrypted object in the bucket.
- Download the object and verify that it opens correctly on the local workstation.



**Figure 15:** Restricted tester user downloading the encrypted object from S3; the file is saved successfully on the local machine.

### Step 5: Attempt Unauthorized Administrative Console Access

- With the restricted user, attempt to navigate to IAM or other account-level administrative sections.
- Confirm that AccessDenied messages appear for these operations.



**Figure 16:** IAM dashboard for the tester account showing multiple AccessDenied findings (e.g., listing MFA devices, access keys, and account aliases).

## 6. Validation and Testing

This section documents how we validated that each security improvement was effective. Wherever relevant, we specify expected vs. actual results and include concrete evidence screenshots.

### 6.1 Validation of Security Group Hardening

#### *Test 1: SSH Access Blocked Externally*

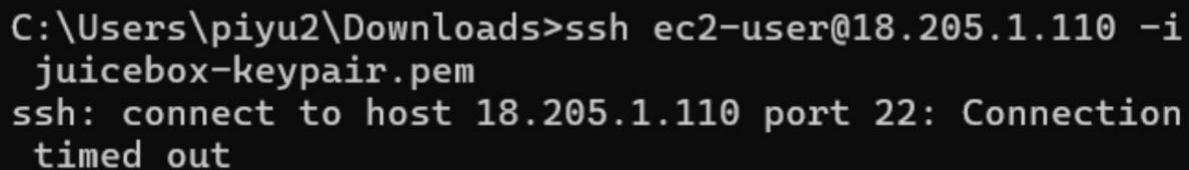
**Goal:** Confirm that SSH (port 22) is no longer reachable from the public internet.

**Procedure:**

- From a local machine, run:  
`ssh ec2-user@<instance-public-ip>`

**Expected Result:** Connection fails with *connection timed out* or *connection refused*.

**Actual Result:** SSH attempts could not establish a connection, confirming that inbound SSH is effectively blocked.



```
C:\Users\piyu2\Downloads>ssh ec2-user@18.205.1.110 -i
juicebox-keypair.pem
ssh: connect to host 18.205.1.110 port 22: Connection
timed out
```

**Figure 17:** SSH connection from a local workstation timing out after Security Group hardening (port 22 blocked).

#### *Test 2: HTTPS Outbound Allowed*

**Goal:** Confirm that outbound HTTPS is permitted so the instance can reach external APIs or package repositories.

**Procedure:**

- From within the EC2 instance (via SSM), run:  
`curl https://www.google.com`

**Expected Result:** HTML content or a 200 OK response.

**Actual Result:** The command returned a successful response, confirming outbound HTTPS is allowed.

```
sh-5.2$ curl https://www.google.com
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="en"><head><meta
content="Search the world's information, including webpages, images, videos and more. Google h
s many special features to help you find exactly what you're looking for." name="description"
<meta content="noodp, " name="robots"><meta content="text/html; charset=UTF-8" http-equiv="Co
tent-Type"><meta content="/images/branding/google/1x/google_standard_color_128dp.png" itemp
op="image"><title>Google</title><script nonce="Em9X-Rv6tybmshq8N_UeIQ">(function(){var _g={kf
:'pWM4afS1BMbZ5NoPuOG6A4',kEXPI:'0,1304224,7,2845681,90133,48791,30022,16105,344796,100066,5
32745,16,3,16,20,36811502,25228681,123988,28395,65168,30631,9138,4600,328,6225,63455,710,1509
,8162,3286,34514,28333,48296,608,24131,6280,7707,7,14749,6055,12581,4719,19385,2126,2863,1,28
2,3088,7010,9747,3705,1978,3605,16008,4493,16551,9920,11976,1,4371,6292,6436,9743,2646,98,5,4
1,1164,3273,8709,5961,5031,9592,4914,792,405,185,1,2,1379,136,3355,196,5814,2,4205,1,4479,580
,1430,4384,2,873,2,2230,5108,3008,25,770,795,3279,4,5393,564,2763,2247,6,5796,2405,2,1400,761
,3321,1031,5337,310,13,522,845,1201,2,11,2,432,1128,3,2842,9,27,8,377,2678,1009,2,1669,5,72,4
2768,2110,5773,3727,1515,1475,312,297,1,1035,82,389,4,298,901,1448,5,677,3527,5,73,537,15,164
570,298,2414,1397,499,1950,2993,1275,7,1784,1004,46,2,295,1619,5,514,666,5367,1251,4,82,1,441
2130,2,4632,736,4,963,95,645,909,4,795,1775,339,2196,756,982,347,245,4,485,497,249,5,438,4,13
```

**Figure 18:** curl https://www.google.com from the hardened instance showing successful HTTPS egress.

### Test 3: HTTPS to AWS Console Allowed

**Goal:** Validate that connections to AWS endpoints are still allowed over HTTPS for operational needs.

#### Procedure:

- From within the instance, run:  
curl https://aws.amazon.com

**Expected Result:** HTML content from the AWS homepage.

**Actual Result:** The request returned the full HTML for the AWS landing page.

```
sh-5.2$ curl https://aws.amazon.com
<!doctype html>
<html lang="en-US" data-static-assets="https://a0.awsstatic.com" class="aws-lng-en_US">
<head>
  <meta http-equiv="Content-Security-Policy" content="default-src 'self' data: https://a0.awss
tatic.com https://prod.us-east-1.ui.gcr-chat.marketing.aws.dev; base-uri 'none'; connect-src '
self' https://*.analytics.console.aws.a2z.com https://*.aws.corrivium.live https://*.harmony.a
2z.com https://*.marketing.aws.dev https://*.panorama.console.api.aws https://*.prod.chc-featu
res.uxplatform.aws.dev https://*.us-east-1.prod.mrc-sunrise.marketing.aws.dev https://112-tzm-
766.mktorep.com https://112-tzm-766.mktoutil.com https://8810.clrt.ai https://a0.awsstatic.co
m https://a0.p.awsstatic.com https://a1.awsstatic.com https://amazonwebservices.d2.sc.omtrdc.n
et https://amazonwebservicesinc.tt.omtrdc.net https://api-v2.builderprofile.aws.dev https://ap
i.regional-table.region-services.aws.a2z.com https://api.us-west-2.prod.pricing.aws.a2z.com ht
tps://artifacts.us-west-2.prod.plc.billing.aws.dev https://assets.storylane.io https://auth.aw
s.amazon.com https://aws.amazon.com https://aws.amazon.com/p/sf/ https://aws.demdex.net https:
//b0.p.awsstatic.com https://c0.b0.p.awsstatic.com https://calculator.aws https://chatbot-api.
```

**Figure 19:** curl https://aws.amazon.com succeeding, confirming allowed HTTPS to AWS endpoints.

*Test 4: Non-Allowed Outbound Port Blocked*

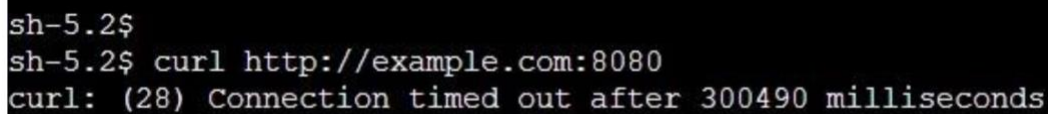
**Goal:** Ensure that outbound traffic on non-permitted ports is blocked.

**Procedure:**

- From within the instance, run:  
`curl http://example.com:8080` or `curl http://1.1.1.1:22`

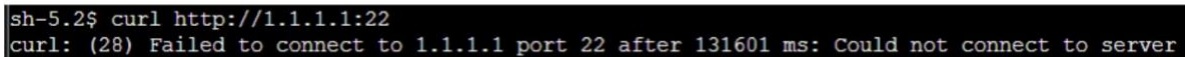
**Expected Result:** Connection fails or times out.

**Actual Result:** Outbound connection attempts on non-allowed ports failed, confirming the outbound restrictions are effective.



```
sh-5.2$  
sh-5.2$ curl http://example.com:8080  
curl: (28) Connection timed out after 300490 milliseconds
```

**Figure 20:** Example of outbound connection on a non-allowed port timing out due to restrictive Security Group and NACL egress rules.



```
sh-5.2$ curl http://1.1.1.1:22  
curl: (28) Failed to connect to 1.1.1.1 port 22 after 131601 ms: Could not connect to server
```

**Figure 21:** `curl http://1.1.1.1:22` failing, demonstrating that arbitrary outbound ports are blocked.

*Test 5: DNS Resolution Still Works*

**Goal:** Validate that DNS queries are still allowed so the instance can resolve external hostnames.

**Procedure:**

- From within the instance, run:  
`nslookup amazon.com`

**Expected Result:** DNS server response with one or more IP addresses.

**Actual Result:** The command returned multiple IP addresses for `amazon.com`, confirming that DNS is functioning through UDP/53 as intended.



The screenshot shows the AWS Management Console interface. At the top, it displays 'Session ID: admin-jv26fu2i52bztqslqb7lhbzkdq' and 'Instance ID: i-0a41e22d7d669e364'. There is a 'Shortcuts' button and a 'Terminate' button. Below this, a terminal window is open, showing the command 'nslookup amazon.com' and its output. The output indicates a successful DNS resolution via the allowed DNS rule.

```
sh-5.2$ nslookup amazon.com
Server:      10.0.0.2
Address:     10.0.0.2#53

Non-authoritative answer:
Name:   amazon.com
Address: 98.82.161.185
Name:   amazon.com
Address: 98.87.170.74
Name:   amazon.com
Address: 98.87.170.71
```

**Figure 22:** nslookup amazon.com from the EC2 instance showing successful DNS resolution via the allowed DNS rule.

## 6.2 Validation of Network ACL Restrictions

### *Test 6: Direct Access to EC2 Blocked by NACL*

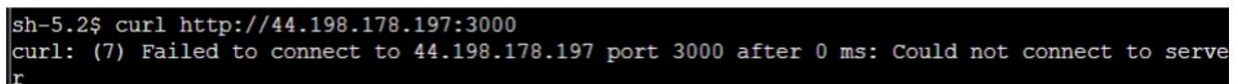
**Goal:** Ensure that the NACL prevents direct access to the EC2 instance, forcing traffic through the intended entry points.

#### **Procedure:**

- From an external client, run:  
`curl http://<instance-public-ip>:3000`

**Expected Result:** Request should fail (timeout or other network error).

**Actual Result:** Direct access to the instance on port 3000 was blocked, confirming the NACL is filtering as intended.



The screenshot shows a terminal window with the command 'curl http://44.198.178.197:3000' and its output. The output indicates that the request failed to connect to the specified IP and port.

```
sh-5.2$ curl http://44.198.178.197:3000
curl: (7) Failed to connect to 44.198.178.197 port 3000 after 0 ms: Could not connect to serve
```

**Figure 23:** curl to the EC2 instance on port 3000 failing, demonstrating that NACL and SG rules block direct access.

### *Test 7: Application Reachability via Approved Path*

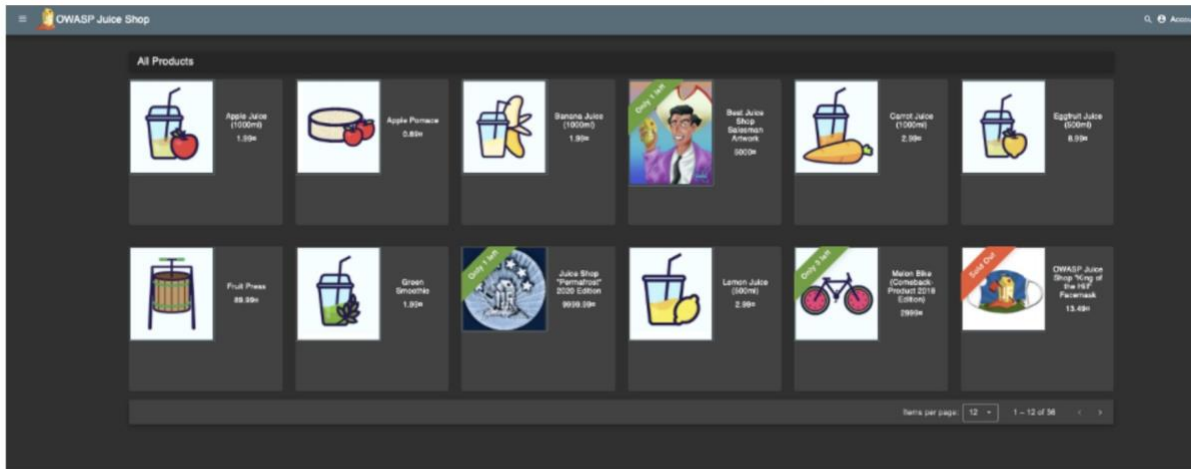
**Goal:** Verify that the application is still reachable via the approved path (ALB endpoint) despite the hardened network controls.

#### **Procedure:**

- Access the Juice Shop application via its ALB DNS name.

**Expected Result:** The Juice Shop home page loads; basic functionality works.

**Actual Result:** Application remained reachable and functional through the intended path.



**Figure 24:** OWASP Juice Shop UI successfully loaded through the Application Load Balancer after all network hardening steps.

### 6.3 Validation of S3 SSE–KMS Encryption Enforcement

#### *Test 9: Bucket Default Encryption Check*

**Goal:** Confirm that the bucket has default SSE–KMS configured.

**Procedure:**

- In the S3 console, open the bucket *Properties* tab.
- Verify that *Default encryption* is enabled with *AWS KMS*.

**Expected Result:** SSE–KMS is enabled, and the designated KMS key is displayed.

**Actual Result:** The bucket's default encryption setting showed AWS KMS with the expected key, as captured earlier in Figure 9.

#### *Test 10: Policy Blocking Unencrypted Uploads*

**Goal:** Ensure that the bucket policy denies non-encrypted uploads.

**Procedure:**

- Attempt to upload a file without specifying encryption from a console session or CLI.

**Expected Result:** Upload fails with AccessDenied due to the bucket policy.

**Actual Result:** The console upload resulted in an explicit Access denied error, confirming that the policy is effective (Figure 14). The CLI test in Figure 12 shows the same behavior.

#### *Test 11: Encrypted Upload Success*

**Goal:** Confirm that correctly encrypted uploads succeed.

**Procedure:**

- From a user with write permissions, run the CLI command with SSE–KMS options.

**Expected Result:** The upload completes successfully and the object appears in the bucket.

**Actual Result:** The encrypted upload succeeded and the object is visible in the bucket listing (Figure 13).

## 6.4 Validation of IAM Least Privilege and Encryption Behavior

### *Test 12: Restricted Login Confirmed*

**Goal:** Ensure that the test IAM user operates with limited permissions.

**Procedure:**

- Log in to the AWS console using the restricted tester account.

**Expected Result:** User can access S3 but has limited overall permissions.

**Actual Result:** Subsequent tests (Tests 13–16) show that permissions are tightly scoped, confirming least privilege for this user.

### *Test 13: Upload Attempt Fails for Restricted User*

**Goal:** Confirm that the restricted user cannot upload objects to the bucket.

**Procedure:**

- Attempt to upload a small test file using the restricted account from the console.

**Expected Result:** Upload fails with AccessDenied due to IAM or bucket policy.

**Actual Result:** Upload failed with Access denied as shown in Figure 14, confirming that write operations are blocked.

### *Test 14: Encryption Metadata Verified by Restricted User*

**Goal:** Ensure that the restricted user can still view encryption metadata on existing objects.

**Procedure:**

- Navigate to an existing object in the bucket.
- View the object properties and confirm it is encrypted with SSE–KMS.

**Expected Result:** The object shows SSE–KMS applied.

**Actual Result:** The object stored in the bucket (Figure 13) is known to have been uploaded using SSE–KMS. The restricted user can see and download it, but cannot alter the encryption or upload new objects.

### *Test 15: Downloading Encrypted Object Succeeds* **Goal:**

Validate that decryption works for authorized users.

**Procedure:**

- Using the restricted account, download the encrypted object.
- Open the downloaded file locally.

**Expected Result:** Download and local file access are successful.

**Actual Result:** The file downloaded and opened without error, as shown in Figure 15, confirming KMS decryption worked for an authorized user.

*Test 16: AccessDenied for Administrative Services*

**Goal:** Confirm that the restricted user cannot perform administrative actions.

**Procedure:**

- Attempt to open IAM or Billing console pages.

**Expected Result:** Pages display AccessDenied messages.

**Actual Result:** Multiple AccessDenied messages for IAM administrative APIs are shown in Figure 16, confirming least privilege is enforced.

## 7. Impact Assessment

### 7.1 Before-and-After Comparison

| Category                | Before Hardening                                                          | After Hardening                                                                                              |
|-------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| SSH Access              | Port 22 open to 0.0.0.0/0, allowing brute-force attacks.                  | SSH completely removed from inbound rules; Session Manager recommended for admin access.                     |
| Application Exposure    | Juice Shop exposed directly on custom port (3000) to all IPs.             | Application reachable only via controlled paths (ALB); direct access to instance restricted via NACL and SG. |
| Outbound Traffic        | All outbound traffic allowed from the instance.                           | Outbound restricted to DNS (53) and HTTPS (443), reducing data exfiltration paths.                           |
| Subnet-Level Security   | Default NACL allowed all inbound and outbound traffic.                    | Custom NACLs enforce explicit allowed ports; all else denied, providing subnet defense-in-depth.             |
| Data Encryption at Rest | No enforcement mechanism for encryption in S3.                            | S3 bucket uses SSE-KMS by default; bucket policy denies non-encrypted uploads.                               |
| Data Access Control     | Potentially broad access to S3; no systematic least-privilege validation. | Restricted IAM tester user enforces least privilege; only read access to encrypted objects verified.         |

**Table 2:** Summary of security posture before vs. after improvements.

### 7.2 Qualitative Security Improvements

#### *Reduced Attack Surface*

By eliminating public SSH and restricting inbound application ports, the opportunities for direct exploitation are significantly reduced. Attackers can no longer trivially attempt brute-force login to the EC2 instance, and direct exposure of the vulnerable application is minimized.

#### *Defense in Depth at the Network Layer*

The introduction of restrictive NACLs adds a second line of defense. Even if a security group were accidentally modified to allow an unwanted port, the NACL would still reject the traffic. This layering is aligned with best practices for cloud network design.

#### *Strong Data Confidentiality Guarantees*

With SSE-KMS and an enforcing bucket policy, there is a clear guarantee that all stored objects in the protected bucket are encrypted. This addresses regulatory and compliance expectations around data-at-rest protection.

#### *Principle of Least Privilege in Practice*

The restricted IAM user, combined with the validation tests, demonstrates that permissions have been narrowed to the minimum required. This lowers the impact of credential compromise and ensures that encryption routines cannot be bypassed by unauthorized actors.

#### *Quantified Improvement to Public Exposure*

**Inbound ports exposed to the public internet were reduced from two (22/3000) to one (80 via ALB only), significantly minimizing externally reachable attack surfaces.**

### 7.3 Trade-Offs and Considerations

#### *Operational Overhead*

Hardening network and access controls introduces additional complexity. For example, administrators must learn to use Session Manager instead of SSH and may need to handle occasional connectivity troubleshooting related to NACL rules. However, this overhead is small compared to the risk reduction.

#### *Performance Impact*

The use of SSE-KMS adds cryptographic operations during object upload and retrieval. In our test environment, the performance impact was negligible, but at very high data volumes there may be cost and latency considerations that should be monitored.

#### *Maintenance of Policies and Keys*

Over time, KMS keys must be rotated, and IAM policies may need to be updated as requirements change. Failure to keep these configurations up to date can introduce availability risks (e.g., if a key is disabled). Our design enables automatic key rotation and uses explicit, auditable policies to minimize misconfiguration risk.

#### 7.4 Residual Risks

Despite the improvements, some residual risks remain:

- The environment still relies on a single EC2 instance (no auto-scaling or multi-AZ redundancy).
- Network misconfiguration remains possible if NACLs and security groups are modified without change control.
- Additional services (e.g., RDS, CloudTrail, GuardDuty) are not fully incorporated in this iteration, leaving some aspects of monitoring and response outside scope.

These residual risks are acceptable within the context of a controlled academic lab, but they point directly to future work.

## 8. Reflection & Lessons Learned

### 8.1 Challenges Faced

Our team encountered several challenges while designing and implementing the hardening measures:

- **Balancing Security and Reachability:** Tightening security groups and NACLs without accidentally locking ourselves out of the environment required careful planning, especially when testing from different networks.
- **Understanding NACL Stateless Behavior:** Because NACLs are stateless, we had to define both inbound and outbound rules for allowed flows. Misconfiguring ephemeral port ranges can easily break connectivity.
- **Coordinating IAM and Encryption:** The interaction between IAM policies, S3 bucket policies, and KMS key policies can be subtle. Ensuring that authorized operations worked, while unauthorized operations failed, required multiple iterations of testing.
- **Evidence Collection and Documentation:** Capturing comprehensive, labeled evidence in the form of screenshots, CLI outputs, and configuration snippets was a non-trivial documentation effort, but it reinforced the importance of reproducibility.

### 8.2 Key Lessons Learned

- **Security Controls Must Be Validated, Not Assumed:** Simply configuring encryption or access controls is insufficient. Validation tests, including negative tests, are essential to proving that controls work as intended.
- **Defense in Depth Is Powerful:** Combining security groups and NACLs provided a strong layered defense. We observed firsthand how subnet-level controls can prevent mistakes at the instance level from turning into exploitable gaps.
- **Least Privilege Is a Continuous Process:** Designing permissions for the test IAM user emphasized that least privilege is iterative. Each test case informed small adjustments to the policies until the intended behavior was achieved.
- **Designing for Recoverability and Future Change:** Using customer-managed KMS keys and explicit bucket policies creates a sustainable foundation. As the system evolves (e.g., new buckets, more services), the same principles can be extended systematically.

### 8.3 Future Improvements and Recommendations

If we had more time and resources, we would extend this work in several directions:

- **Move Database to RDS with Multi-AZ:** Migrating the local database to Amazon RDS with Multi-AZ deployment and built-in encryption would strengthen both availability and data protection.
- **Introduce Centralized Monitoring and Alerting:** Enabling AWS CloudTrail, GuardDuty, and Security Hub would improve detection of anomalous activity and provide audit trails for incident response.
- **Automate Infrastructure with IaC:** Expressing security groups, NACLs, S3 policies, and IAM roles in Infrastructure-as-Code (e.g., CloudFormation or Terraform) would make it easier to reproduce and version-control the environment.
- **Formalize Change Management:** Implementing a change review process for network and IAM changes would further reduce the risk of misconfigurations reintroducing vulnerabilities.

Overall, this project reinforced that hardening cloud environments is not a one-time activity but an ongoing lifecycle. The improvements documented here move our environment significantly closer to a secure, resilient architecture while leaving a clear roadmap for future enhancements.